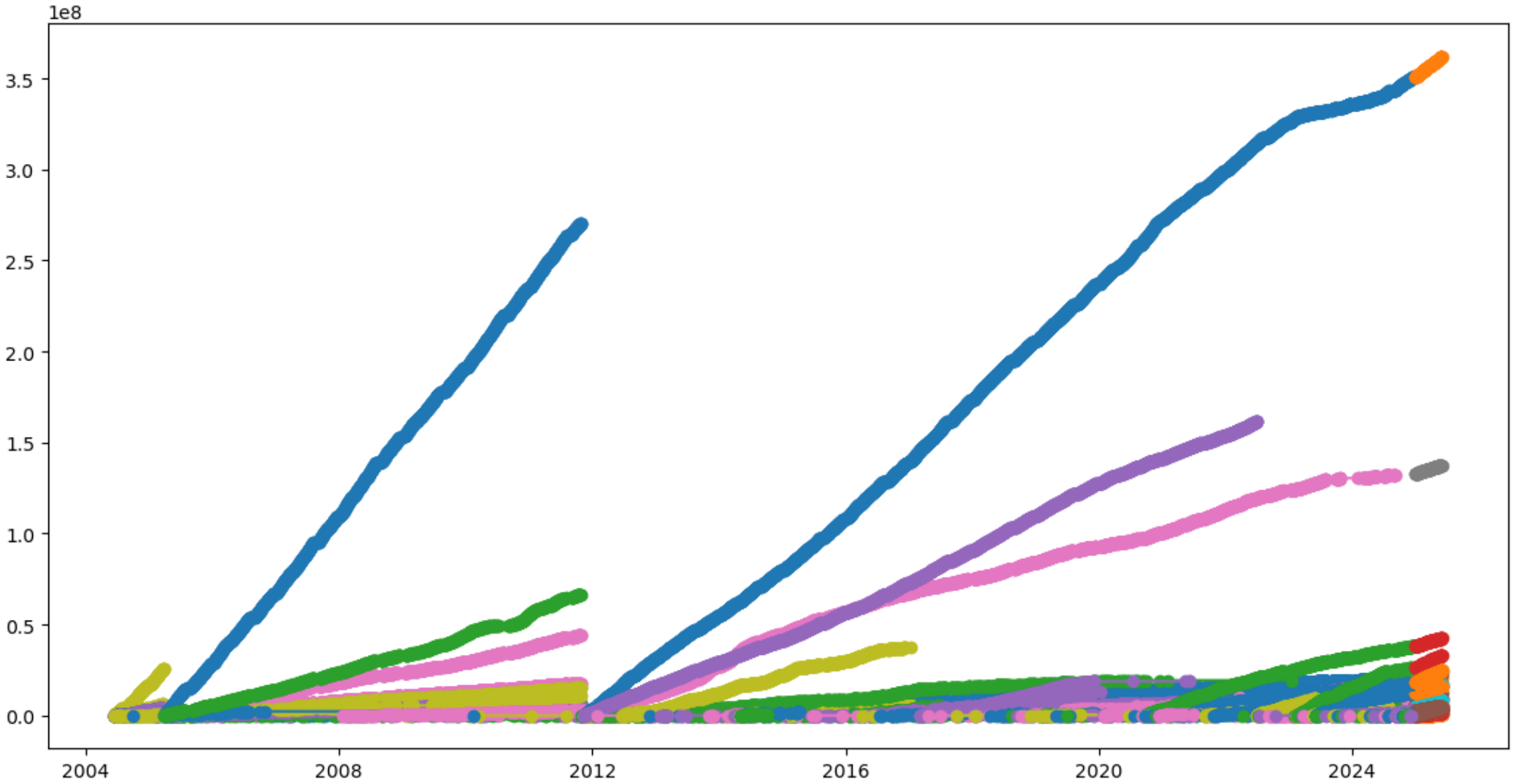
TDT 4173

Modern Machine Learning in Practice

Group 74

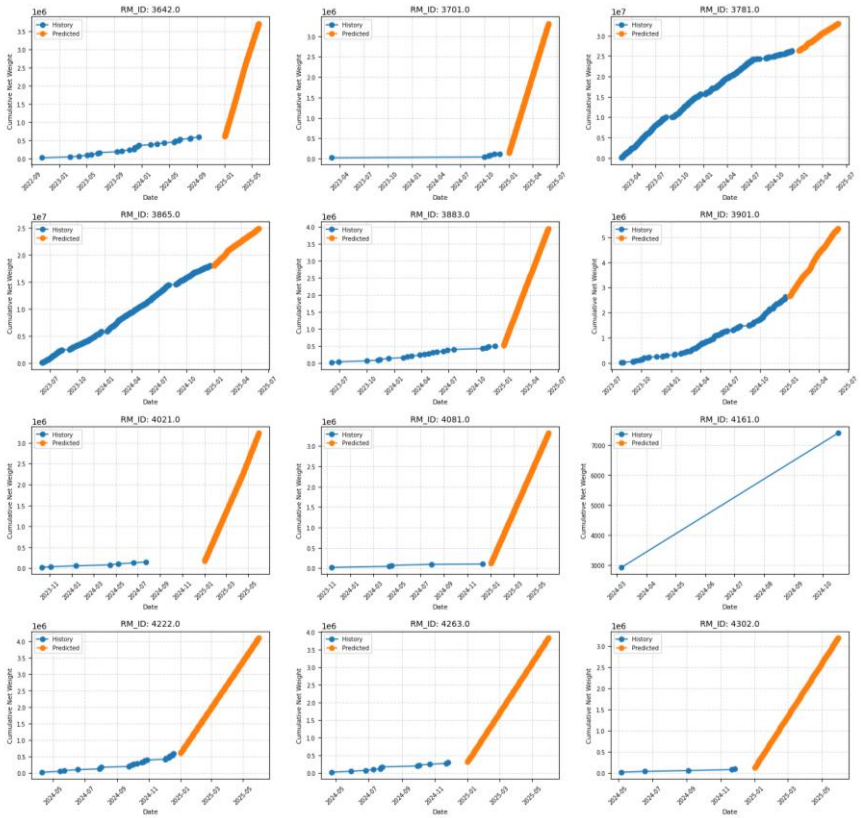
Hanna, Sambavi and Siri

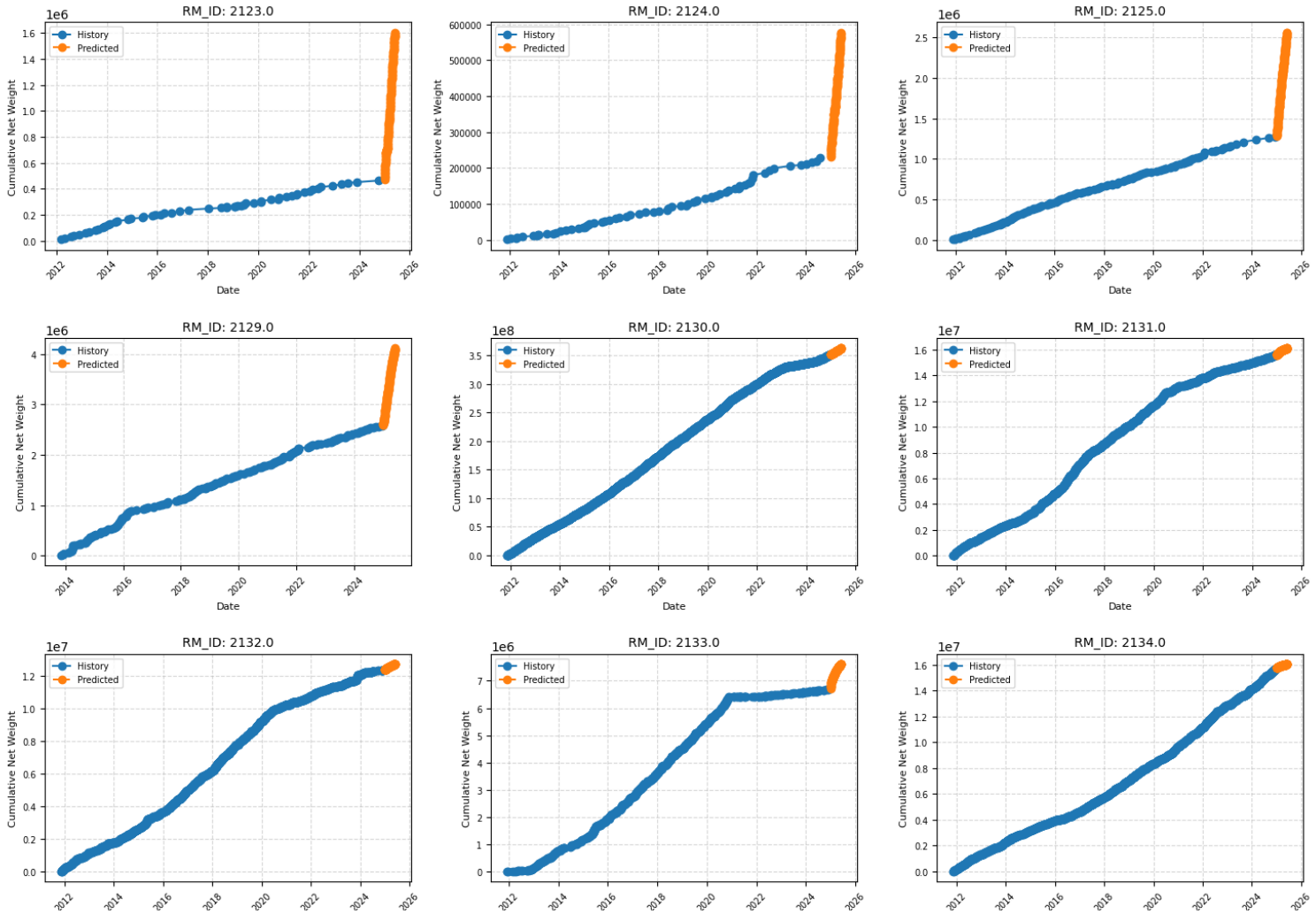
Started by doing quick weight and date check



Tried predicting

- Used only XGBoost

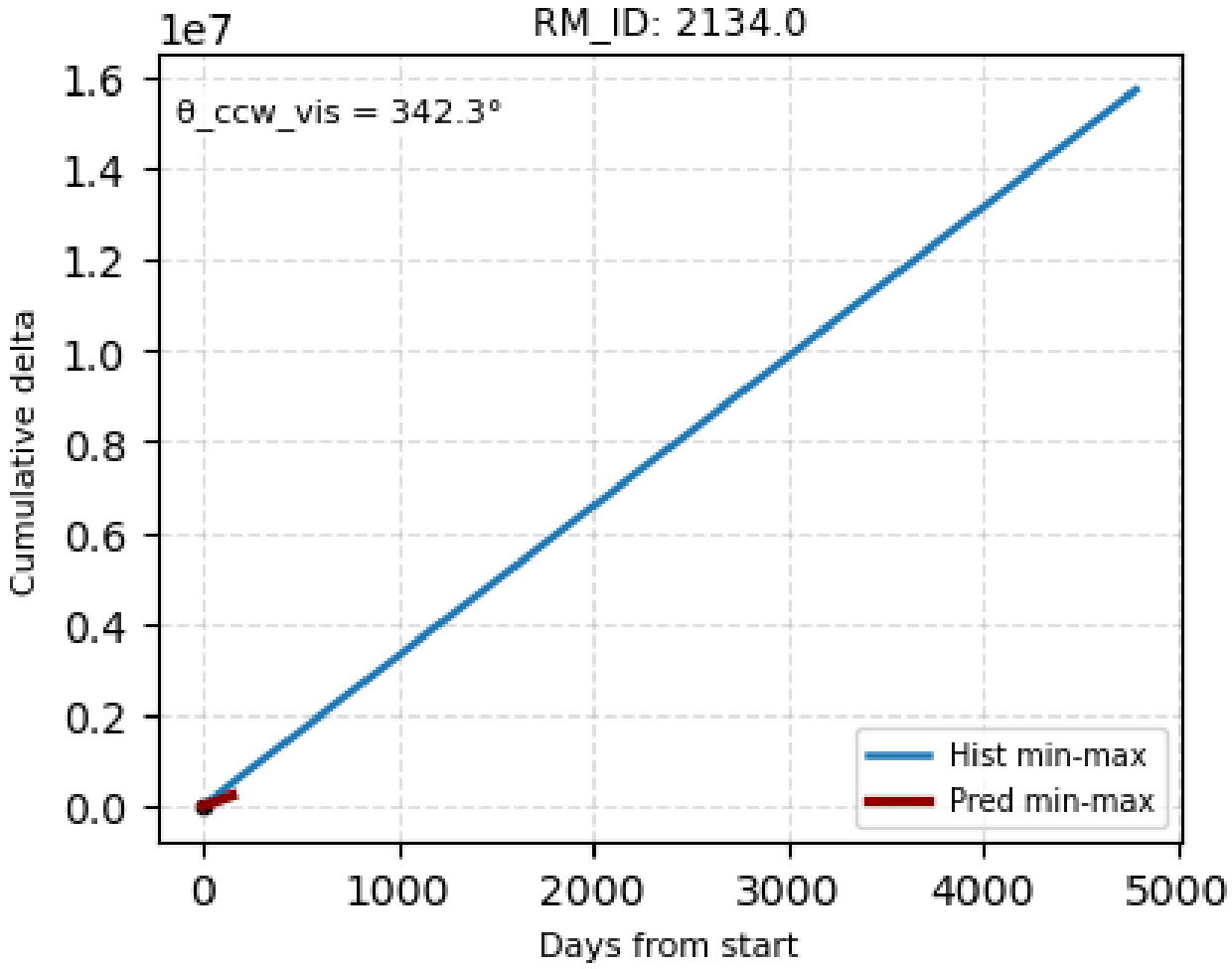


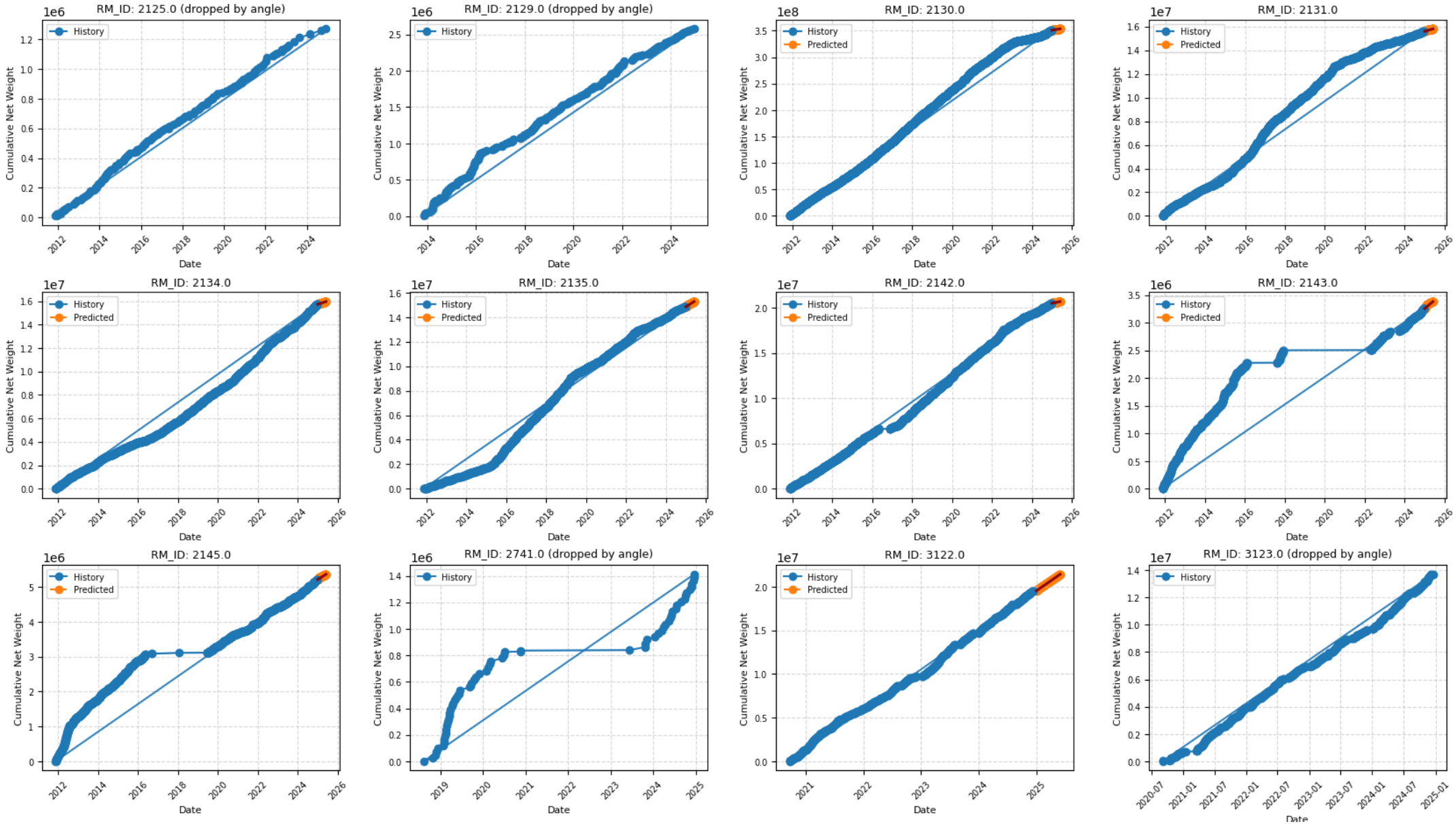


Super Wrong!

Super simple angle check

- One line from first point of historic data to last point in historic data set (blue)
- Another from first point in predicted data to last point in predicted data (red)
- If angle between 0-10 and 330-360 degrees
- Plot everything!



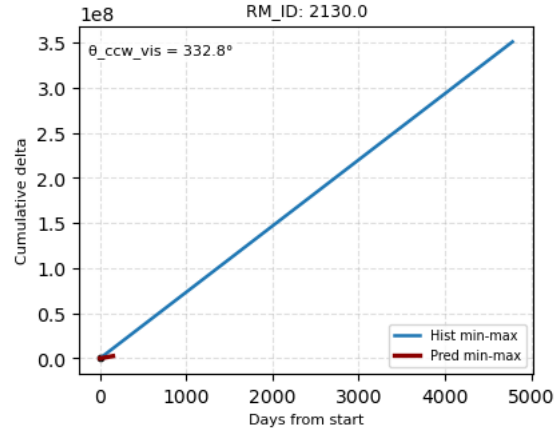

RM_ID: 2125.0 (dropped by angle)
RM_ID: 2129.0 (dropped by angle)
RM_ID: 2130.0
RM_ID: 2131.0
RM_ID: 2134.0
RM_ID: 2135.0
RM_ID: 2142.0
RM_ID: 2143.0
RM_ID: 2145.0
RM_ID: 2741.0 (dropped by angle)
RM_ID: 3122.0
RM_ID: 3123.0 (dropped by angle)
History
Predicted
Cumulative Net Weight
Date

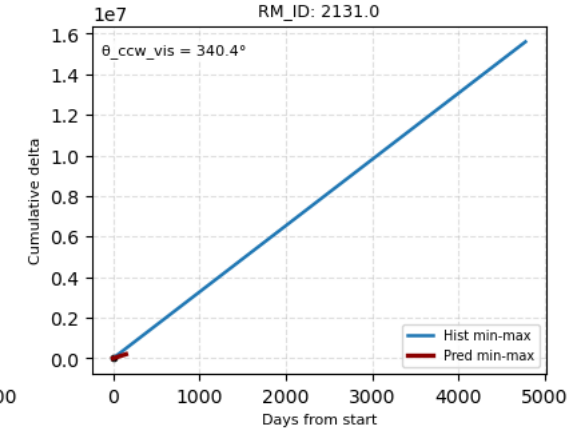
RM_ID: 2125.0 (dropped)

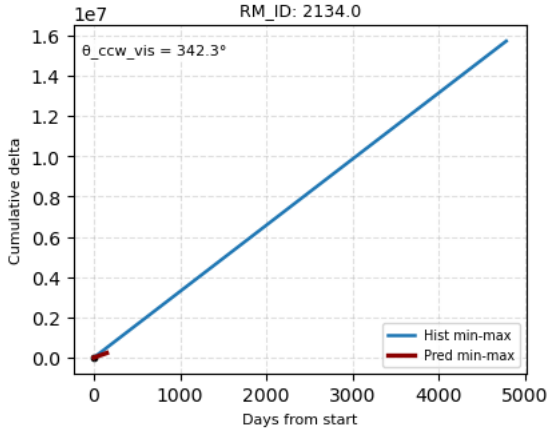
dropped by angle

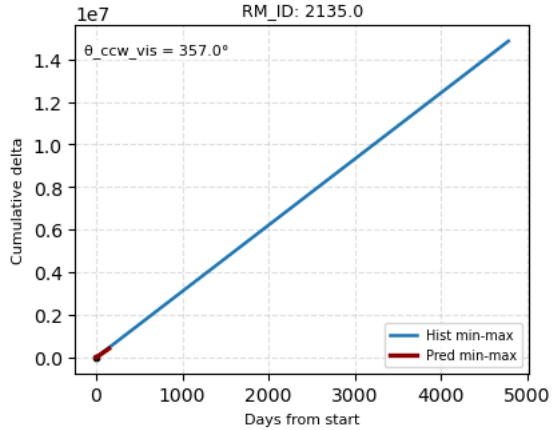
RM_ID: 2129.0 (dropped)

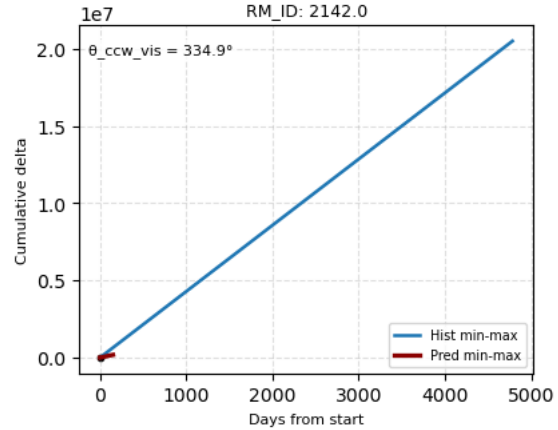
dropped by angle

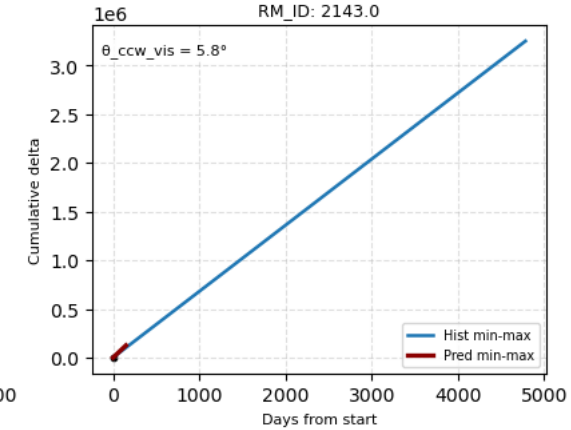


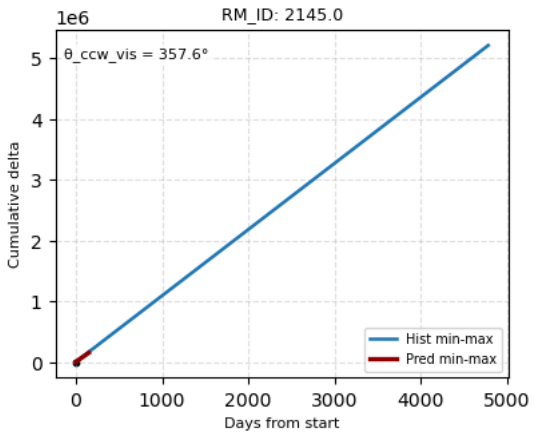






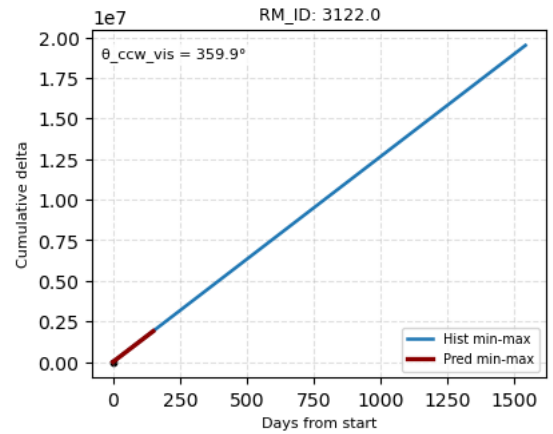






RM_ID: 2741.0 (dropped)

dropped by angle



RM_ID: 3123.0 (dropped)

dropped by angle

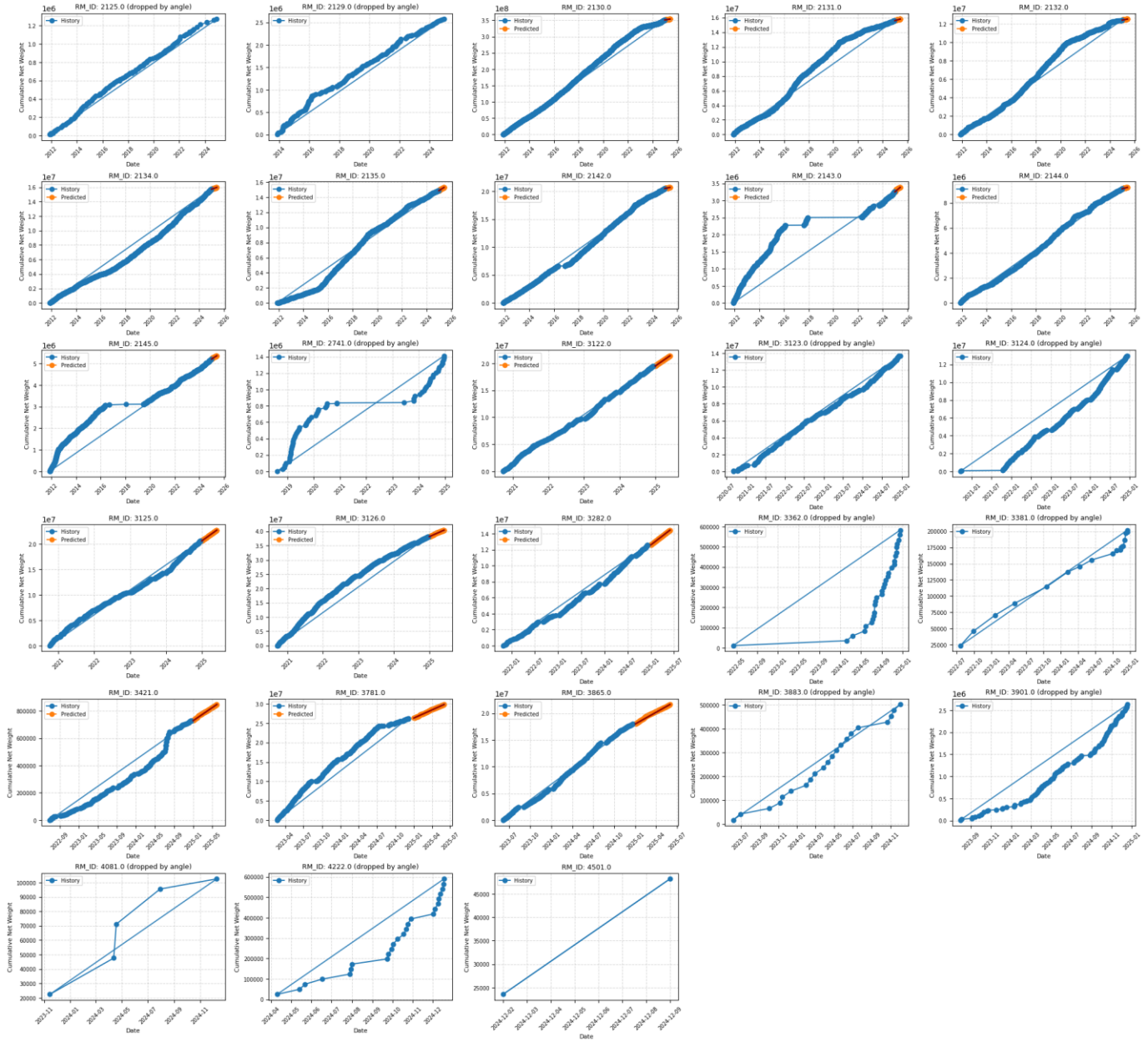
Code – Angle check

- Struggled a lot to get this right
- Have used this throughout the whole process

```
    # transform endpoints to display coordinates
    p0_disp = ax_tmp.transData.transform((0.0, 0.0))
    p_h_disp = ax_tmp.transData.transform((hx1, hy1))
    p_p_disp = ax_tmp.transData.transform((px1, py1))

    v_h_disp = np.array([p_h_disp[0] - p0_disp[0], p_h_disp[1] - p0_disp[1]], dtype=float)
    v_p_disp = np.array([p_p_disp[0] - p0_disp[0], p_p_disp[1] - p0_disp[1]], dtype=float)

    if np.linalg.norm(v_h_disp) > 0 and np.linalg.norm(v_p_disp) > 0:
        cross = v_h_disp[0]*v_p_disp[1] - v_h_disp[1]*v_p_disp[0]
        dot = v_h_disp[0]*v_p_disp[0] + v_h_disp[1]*v_p_disp[1]
        ang_rad = np.arctan2(cross, dot)
        ang_ccw = ang_rad if ang_rad >= 0 else (ang_rad + 2*np.pi)
        ang_deg = float(np.degrees(ang_ccw))
    else:
        ang_deg = 0.0

    angles_list.append({"rm_id": rm, "theta_ccw_vis_deg": ang_deg})

    # drop prediction when visual angle is strictly between 10 and 300
    if 10.0 < ang_deg < 340.0 and rm not in SPARSE_RMS:
        rm_to_drop.add(rm)
else:
    # insufficient points — keep for now but log zero angle
    angles_list.append({"rm_id": rm, "theta_ccw_vis_deg": None})
```

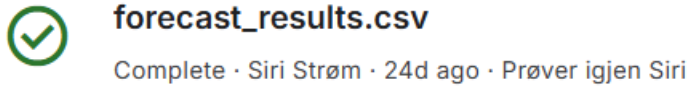
RM_IDs

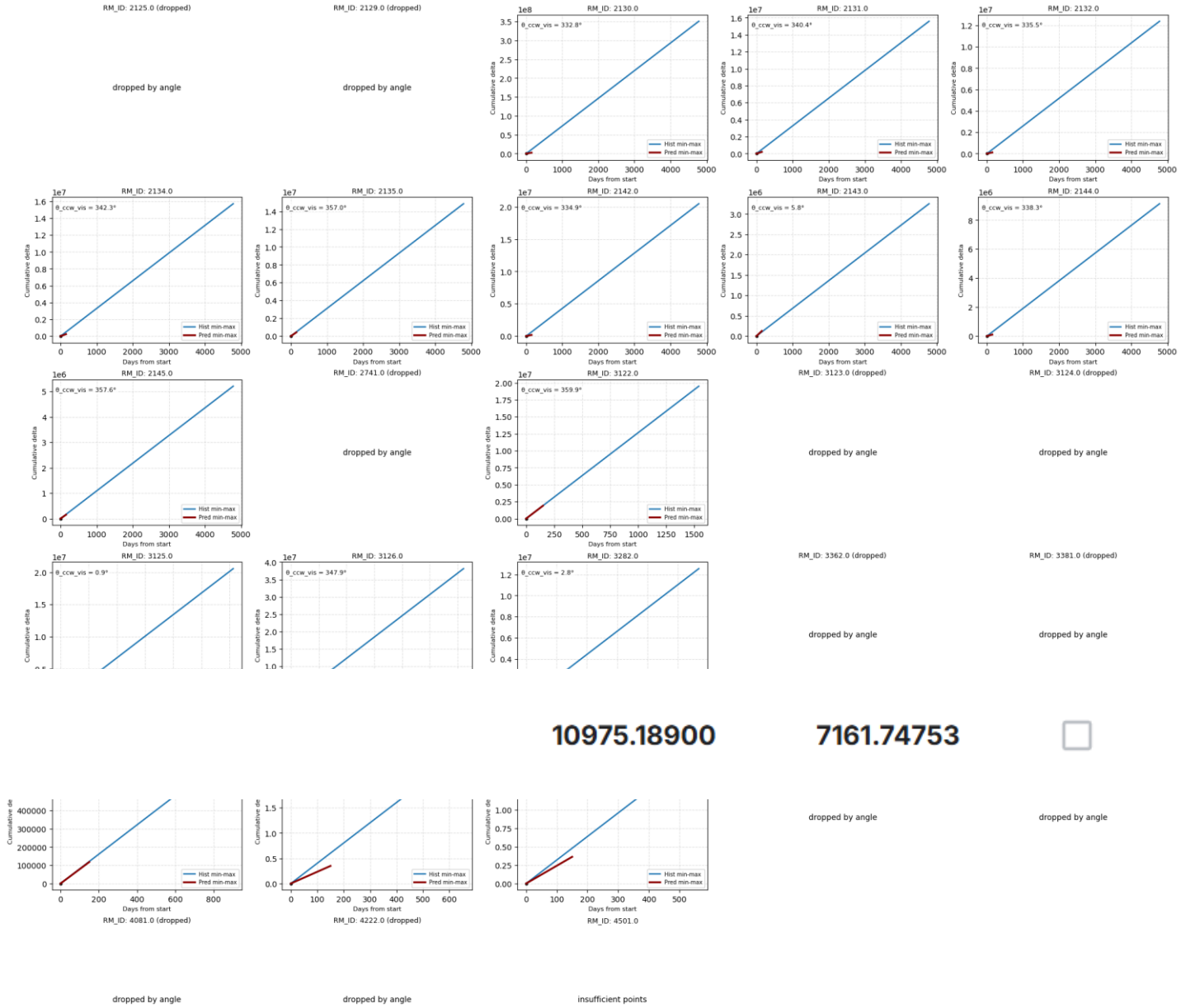
- We ended up with a little under 30 good enough RM_IDs to try and predict



RM_IDs

- But only 16 predictions with the angle check
- Gave good result on Kaggle, seemed we were on the right track





Next steps, predict more RM_IDs

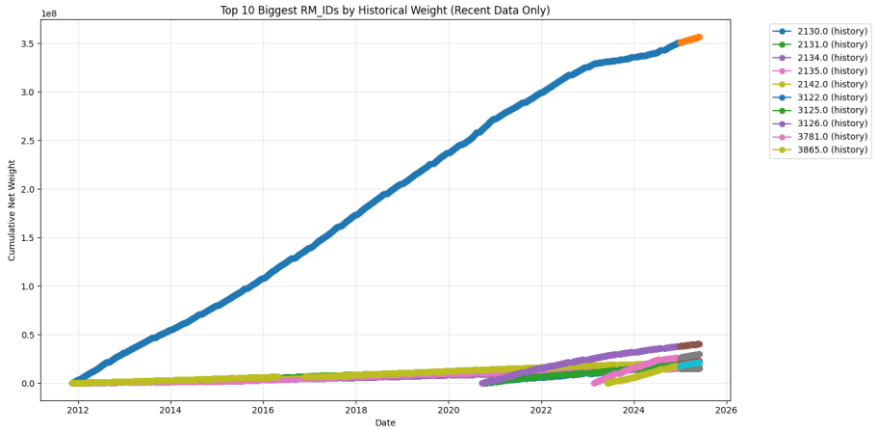
- Next we tried to predict the rest of the RM_IDs
- Did second pass with LightGBM
- See now: helped a lot on public, not on private

Next steps, predict more RM_IDs

- Then we got a bit stuck
- Heard from other groups higher than us: Use purchase orders
  - Tried for almost two weeks without results
  - Changed models, weights etc.
- Ended up going back to just recievals

Went back to the beginning

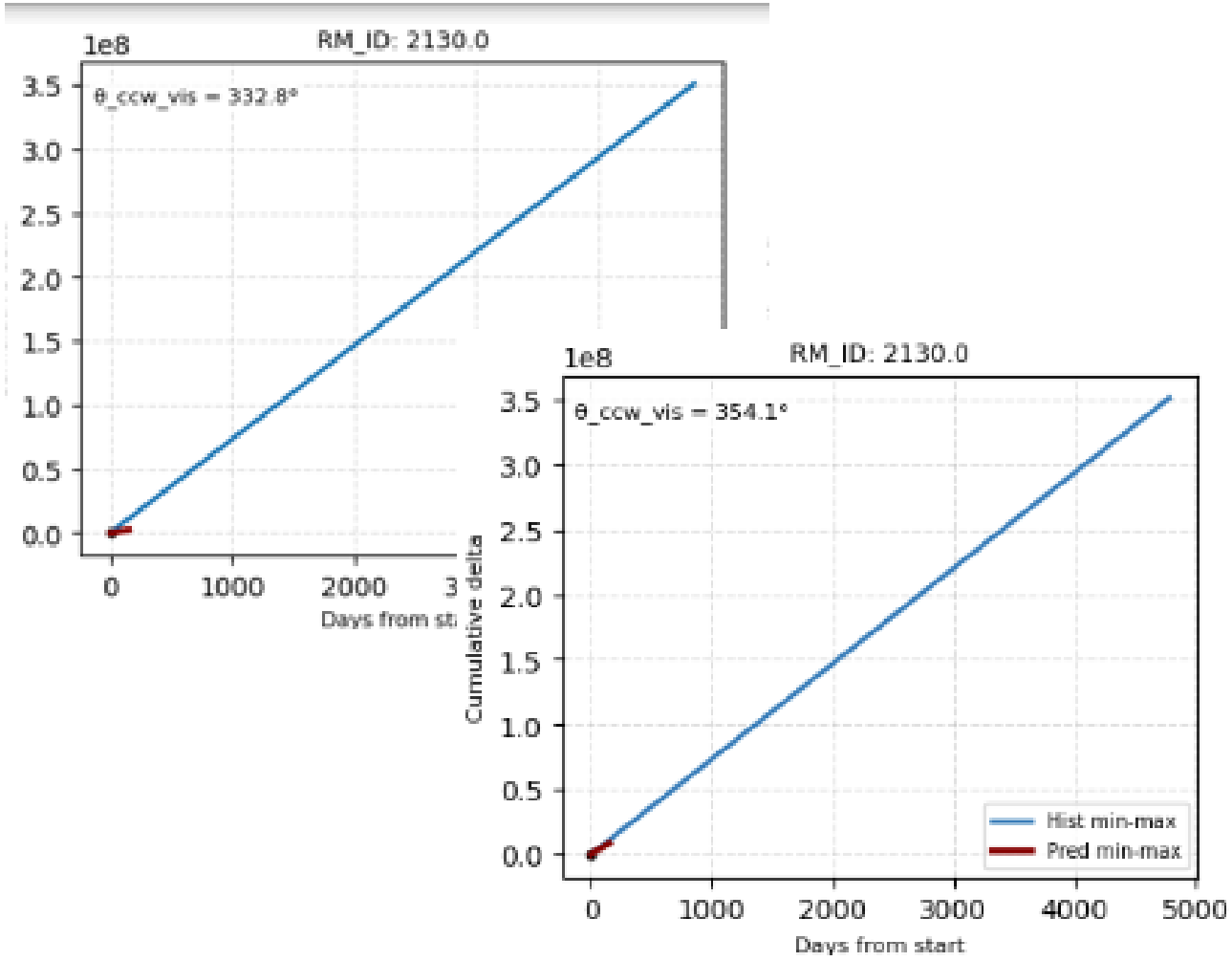
- What could make our score go down?
- Thought that it would be smart to get a prediction for as many rm_ids as possible
  - Third pass using Linear Regression
- Saw that one rm_id was much heavier than the rest
  - 2130




```python
# features / target
X = rm_data[['dayofyear', 'month', 'year', 'weekday']].copy()
y = rm_data['net_weight'].copy()

# zero-out sparse predictors
if len(rm_data) < 10:
    X[['month', 'weekday']] = 0
if rm_data['date'].min() < pd.Timestamp('2020-01-01'):
    X[['year']] = 0

# build future features
future_df = pd.DataFrame({'date': pd.to_datetime(future_dates)})
future_df['dayofyear'] = future_df['date'].dt.dayofyear
future_df['month'] = future_df['date'].dt.month
future_df['year'] = future_df['date'].dt.year
future_df['weekday'] = future_df['date'].dt.weekday
X_future = future_df[['dayofyear', 'month', 'year', 'weekday']]

# handle constant target
if np.isclose(y.std(), 0.0) or y.nunique() <= 1:
    const_pred = float(y.mean()) * 0.40
    y_pred = np.full(len(X_future), const_pred, dtype=float)
else:
    # Linear Regression model
    model = LinearRegression()
    model.fit(X, y)
    y_pred = model.predict(X_future)

    # apply conservative shrink
    y_pred = y_pred * 0.4

    # clip negative predictions to zero
    y_pred = np.maximum(y_pred, 0.0)

# optional: angle logging
```

2130

- Weight super important because 10^8 x 0.2 or 0.8 -> really big error
- Saw from plot underpredicting
  - 333 degrees
- Asked Copilot to help
  - Less conservative
  - Now 354 degrees
- This + linear regression worked!



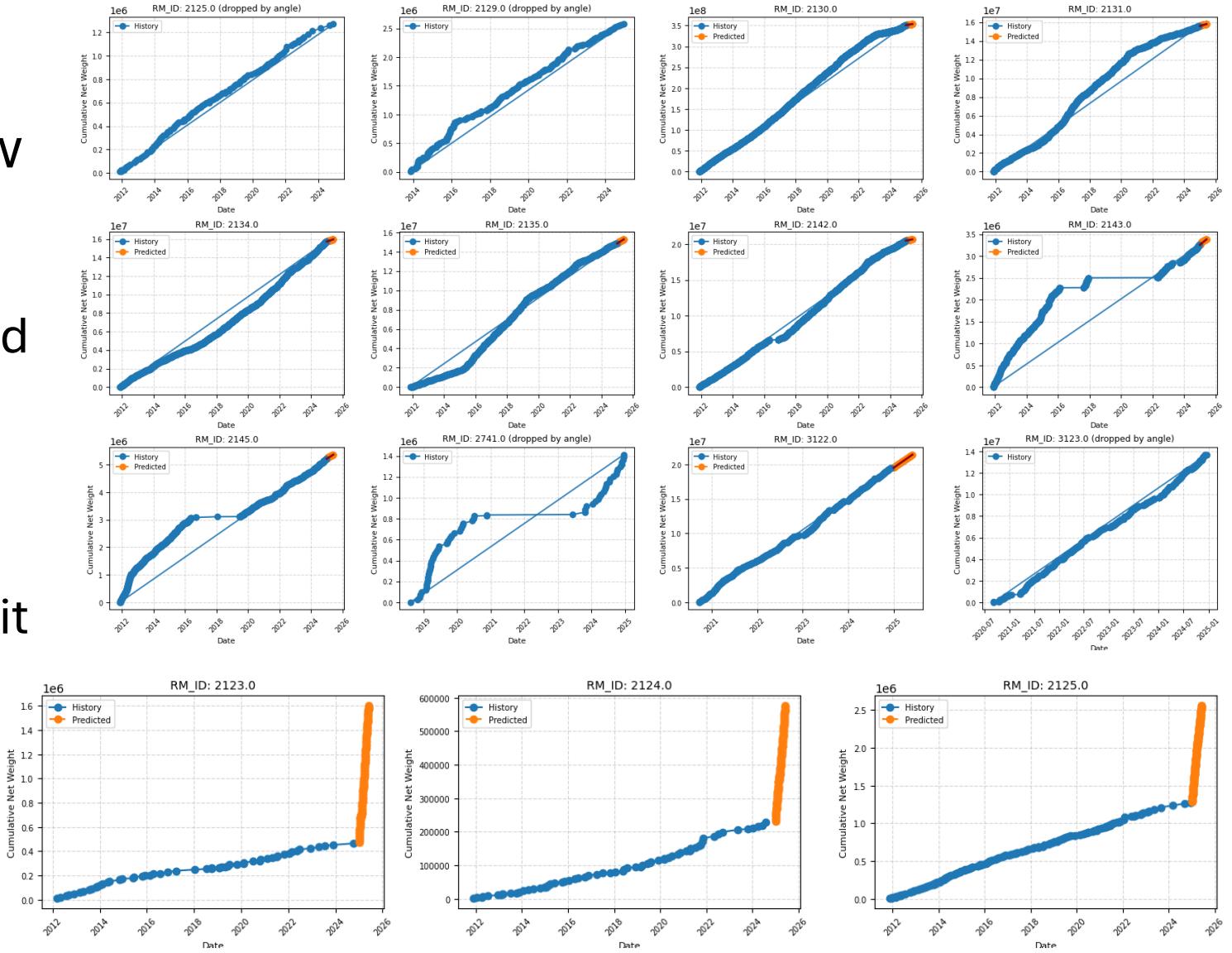
short_notebook_1.csv

Complete · Hanna Stuørd Dale · 3d ago

6736.13293 4768.35296

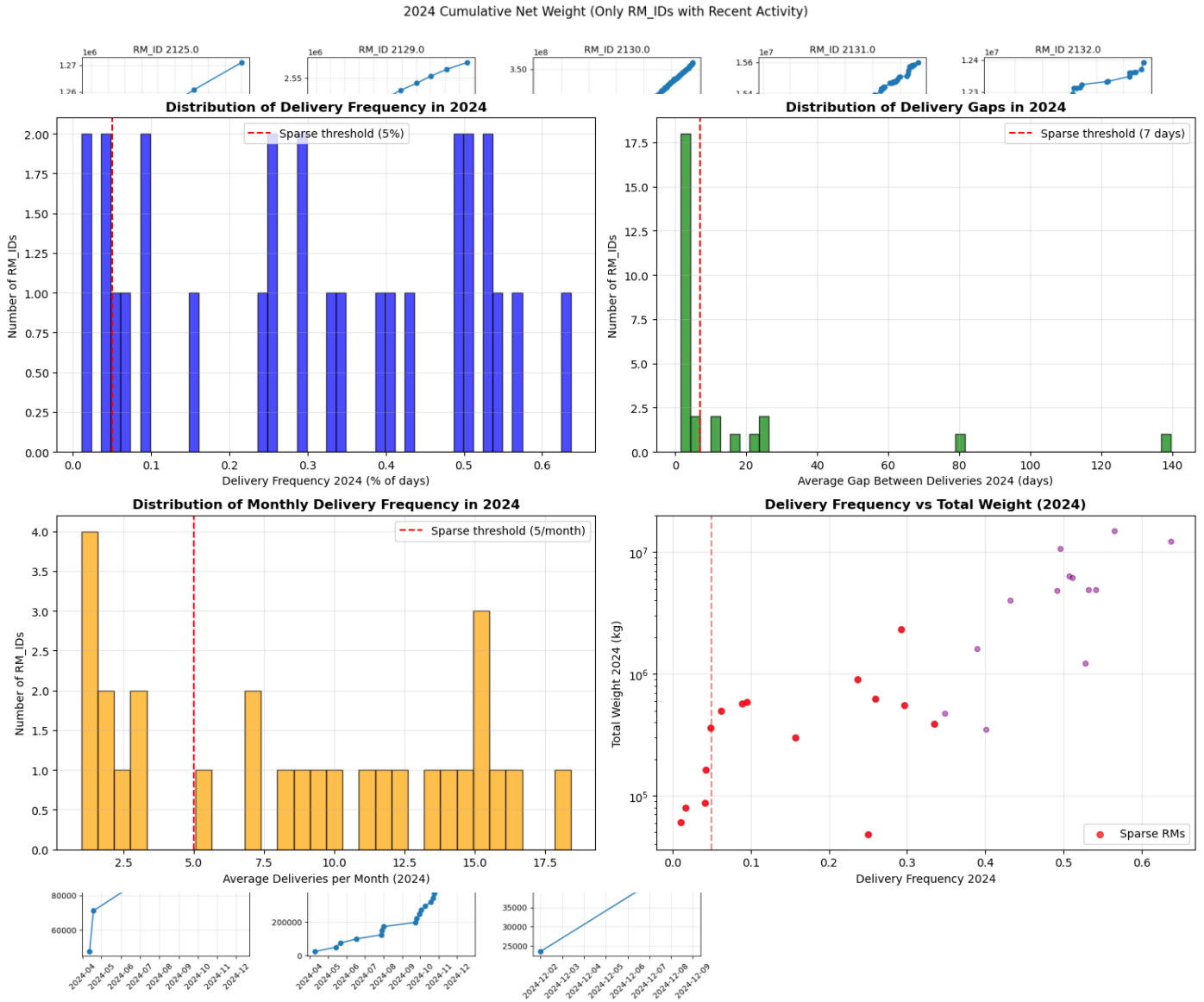
The final stretch

- Kinda lost on what to do now
  - Had just beaten VT 1 on Kaggle, felt like we had to try something else for our second submission
- Why were some predictions really bad?
  - 2125 for example looked like it should have gotten a good prediction



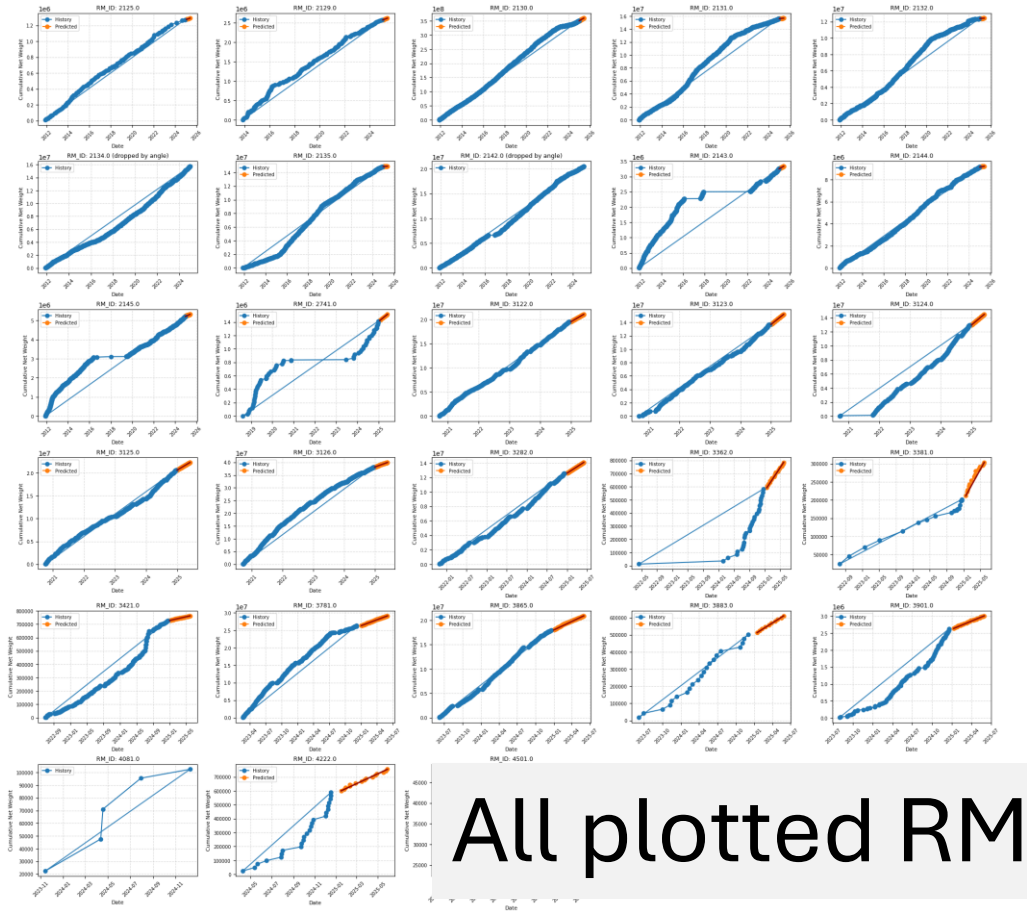
Sparse data

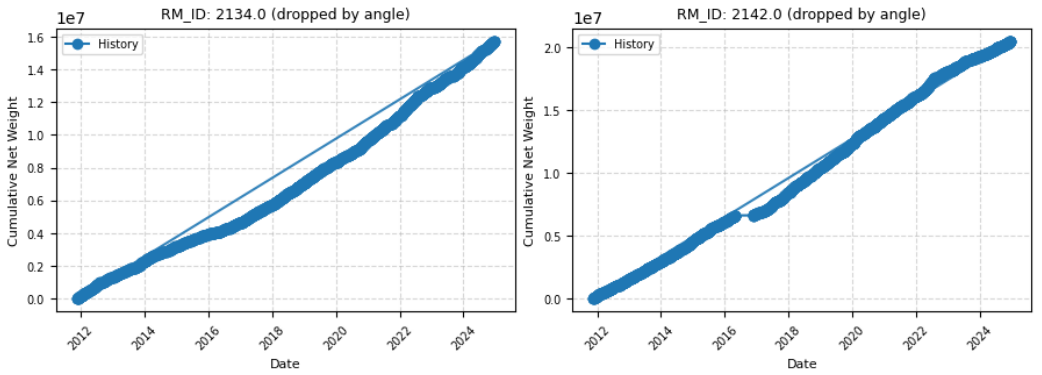
- Looked at only 2024, a lot of the graphs that were badly predicted were sparse
- Decided to do a sparsity analysis
  - How often each rm_id was delivered across 2024
- Quite a lot of the rm_ids were sparse
- Cutoff at 10 deliveres a month on average
- Went down from 151 predictions total to 50 or as few as 5 total predictions for the whole period

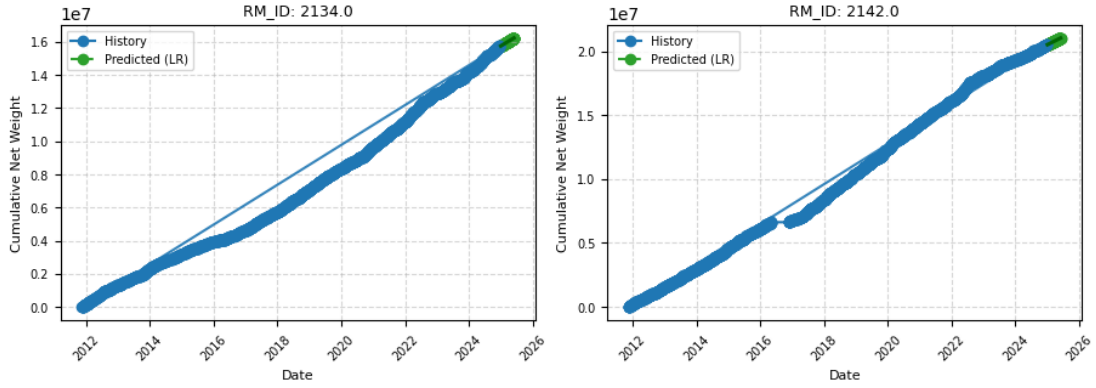


Plots with 3 passes + sparse data

- Ended up dropping angle check on the sparse data







All plotted RM_Ids had non-zero predictions!

- Gave us a score 6400 on Kaggle public leaderboard
  - Thought it was worse than our earlier
- But we had tried something else -> 2. delivery

- If we had more time could definitely try to make this model better
- Overall very proud of how far we came and our solutions